



ECSE 4230: Embedded Systems I

Seven Segment Display Clock & Timer

Derrick Cannon, Ian Garrison, and Dawson Gulasa

01/28/26

Table of Contents

Table of Contents	1
Project Information	2
Group Roles and Responsibilities:	3
GitHub Repository Link:	3
Errors and Modifications to Assignment Instructions.....	3
Deliverable 1: Photo of Hardware Setup.....	3
Deliverable 2: Our Final Timing Function Described	4
Deliverable 3: Characterization of the Timing Functions (With Calculations).....	7
Manual:	8
Calibration Procedure:.....	8
Results:	9
Time.Sleep Characterization:.....	10
Results:	10
Deliverable 4: Final Timing Function Accuracy (2 Hour Test)	11
Link To Timelapse:	12
Test Procedure:	12
Deliverable 5: Sleep Timing Function Accuracy (2 Hour Test)	13
Link to timelapse:.....	14
Test Procedure:	14
Deliverable 6: Comparison between Sleep Timing and Our Timing Function	16
Manual	17
Time.Sleep	17
Direct Comparison Summary:	17
Deliverable 7: Causes of Timing Function Errors:.....	19
Deliverable 8: Main Loop & FSM Description	21
IDLE State	21
AUTO State	21
MANUAL State	22
Overall Loop Behavior	22
	2

Deliverable 9: Description of Debouncing Technique 22

Deliverable 10: Most Challenging Part of The Clock Project 23

References..... 24

Project Information

Group Roles and Responsibilities:

- **Derrick Cannon:**
 - Wrote Python source code, built the physical circuit, and documented results.
- **Ian Garrison:**
 - Characterized timing function, wrote Python source code, and documented results.
- **Dawson Gulasa:**
 - Wrote Python source code, built the physical circuit, and documented results.

GitHub Repository Link:

Below is the link to the GitHub repository containing the following source files:

https://github.com/Herring-UGAECSE-4230-S26/Group7/blob/main/Python/SSD_Clock_Timer/full_clock_module.py

In case of access issues below is an additional reference to our python scripts:

https://drive.google.com/file/d/13ZGIW6nF-1b55iPFjnaZaH_46S3VAOT8/view?usp=sharing

Errors and Modifications to Assignment Instructions

Overall, the assignment instructions were clear and sufficient for completing the project successfully. The hardware requirements, functional expectations, and deliverables were well defined.

The only minor difficulty came with the documentation. On the last assignment we got a lot of points off because we didn't explicitly answer the deliverables, which is understandable. But the deliverables for this project ask us to compare our results in 3 separate bullet points, as well as explain our characterization in 2 separate bullet points. We just wish the bullets were explicit deliverables that needed answering instead of word dumping the same deliverable multiple times. It made doing the document more difficult because we really wanted to ensure we did it correctly this time and follow document expectations.

Deliverable 1: Photo of Hardware Setup

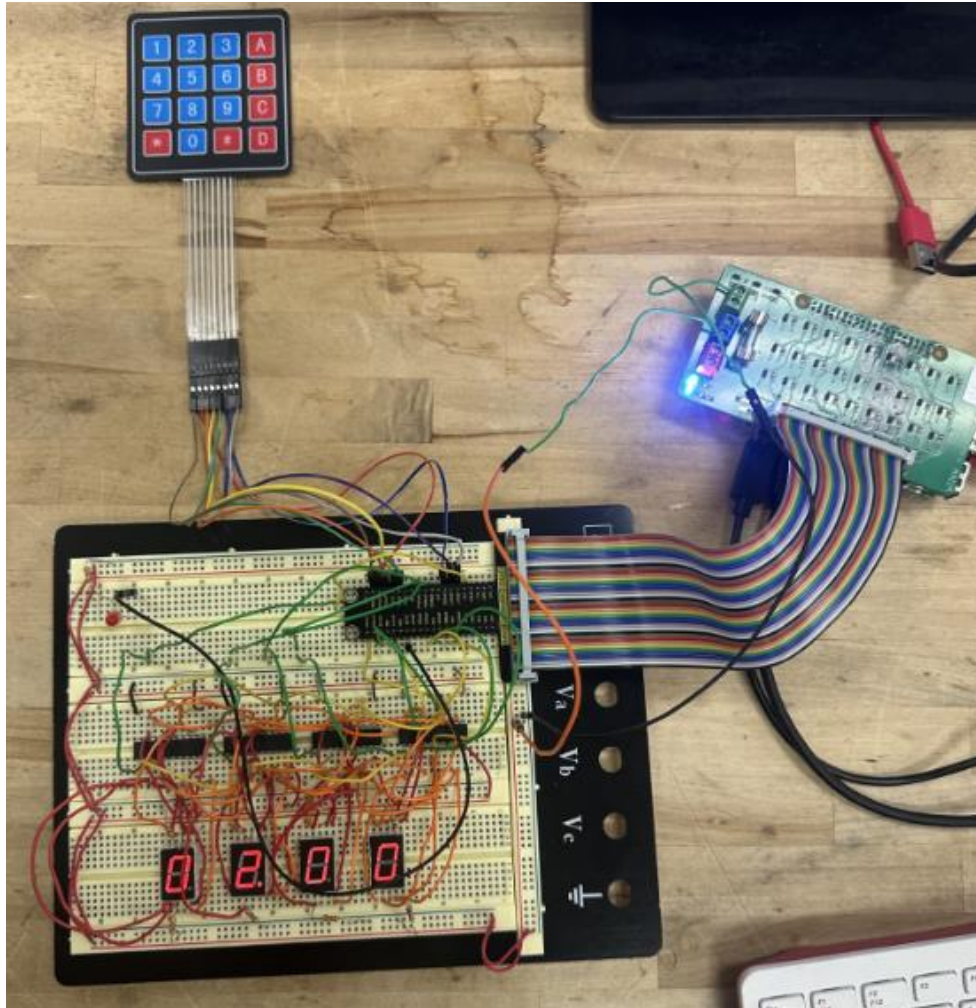


Figure 1: Picture of our finalized hardware setup

Deliverable 2: Our Final Timing Function Described

```
# Busy-loop calibration constant used by manual_tick_nonblocking()
ITERATIONS_PER_SECOND = 2_600_000 # calibrated value (final)

# Busy loop used for timing calibration.
def burn_iterations(n: int):
    i = 0
    while i < n:
        i += 1

# Advance the manual clock using small time slices (busy loop),
# updating the display once per minute (matches existing behavior).
def manual_tick_nonblocking(step_iters: int = 120_000):
    global manual_iter_accum, manual_s, manual_m, manual_h24, pm

    burn_iterations(step_iters)
    manual_iter_accum += step_iters

    # When enough iterations have accumulated to equal 1 second,
    # advance the manual clock by one second.
    while manual_iter_accum >= ITERATIONS_PER_SECOND:
        manual_iter_accum -= ITERATIONS_PER_SECOND

        manual_s += 1
        if manual_s >= 60:
            manual_s = 0
            manual_m += 1
            if manual_m >= 60:
                manual_m = 0
                manual_h24 = (manual_h24 + 1) % 24

        # Update display once per minute
        h12, pm_flag = convert_24h_to_12h(manual_h24)
        pm = pm_flag
        load_12h_into_memory(h12, manual_m)
```

Figure 2: Code Snippet of Relevant Timing Logic.

```
elif state == "MANUAL": # User enters time
    if not manual_done:
        manual_entry_step(k)
        if display_on:
            render_display_blink(pm=pm)
        else:
            render_display(pm=False)
    else:
        manual_tick_nonblocking(step_iters=120_000)
        if display_on:
            render_display(pm=pm)
        else:
            render_display(pm=False)

    sleep(0.02)
```

Figure 3: Proof that this function is used in our MANUAL state.

Our final timing function for MANUAL mode is implemented in *manual_tick_nonblocking()*. Instead of calling `sleep(1)`, we approximate one second of elapsed time by executing a calibrated number of CPU loop iterations.

How the timing loop works (step-by-step):

1) Calibration Constant:

ITERATIONS_PER_SECOND represents how many iterations of our busy loop correspond to approximately one real second on our Raspberry Pi while running the full program (GPIO updates, keypad scanning, display refresh, etc.)

2) Chunked iterations (“nonblocking”):

Each pass through the main program calls:

```
burn_iterations(step_iters)
```

This burns a small chunk of iterations rather than trying to burn a whole second at once. This keeps the program responsive.

3) Accumulator-based timing:

After burning *step_iters*, we add it to *manual_iter_accum*. When:

```
manual_iter_accum >= ITERATIONS_PER_SECOND
```

We treat that as one elapsed second:

- Subtract ITERATIONS_PER_SECOND from the accumulator to reset
- Increment *manual_s* (the second counter)

4) Time rollover logic:

The function handles normal clock rollovers:

- If it reaches 60 seconds, increment minutes.
- If minutes reaches 60, increment hours (mod 24 for 12 hour time)

5) Display update policy

The display digits (*display_memory*) are updated once per minute (when seconds roll over), not every second. That reduces display/GPIO overhead while still keeping the time correct.

Where it runs in the program:

This timing function is called only after manual entry is complete (`manual_done == True`) inside the MANUAL state. This shows that the timing loop is the mechanism that advances time during MANUAL operation.

Deliverable 3: Characterization of the Timing Functions (With Calculations)

Manual:

```
# Busy-loop calibration constant used by manual_tick_nonblocking()
ITERATIONS_PER_SECOND = 2_600_000 # calibrated value (final)

# Busy loop used for timing calibration.
def burn_iterations(n: int):
    i = 0
    while i < n:
        i += 1

# Advance the manual clock using small time slices (busy loop),
# updating the display once per minute (matches existing behavior).
def manual_tick_nonblocking(step_iters: int = 120_000):
    global manual_iter_accum, manual_s, manual_m, manual_h24, pm

    burn_iterations(step_iters)
    manual_iter_accum += step_iters

    # When enough iterations have accumulated to equal 1 second,
    # advance the manual clock by one second.
    while manual_iter_accum >= ITERATIONS_PER_SECOND:
        manual_iter_accum -= ITERATIONS_PER_SECOND

        manual_s += 1
        if manual_s >= 60:
            manual_s = 0
            manual_m += 1
            if manual_m >= 60:
                manual_m = 0
                manual_h24 = (manual_h24 + 1) % 24

        # Update display once per minute
        h12, pm_flag = convert_24h_to_12h(manual_h24)
        pm = pm_flag
        load_12h_into_memory(h12, manual_m)
```

Figure 4: Screenshot of Relevant Functions & Variables for Characterization.

Our manual clock mode does not use *sleep(1)* to create a one-second timing. Instead, we implemented a calibrated busy-loop timing approach using a constant *ITERATIONS_PER_SECOND* variable. The design is explained in more detail above.

We originally calibrated the iteration count in a separate timing script. However, once the timing logic was moved into the full integrated program (keypad scanning, display refresh, blinking logic, etc.), the execution count required to approximate one real second changed drastically. Therefore, we characterized/calibrated the constant while running the full clock program, so the measured timing includes all real program overhead.

Calibration Procedure:

1. Enter MANUAL mode and input a known starting time.
2. On the final digit entry (completion of MANUAL entry), start a stopwatch.
3. Stop the stopwatch exactly when the displayed minute rolls over (e.g. from MM to MM+1)
4. Record the measured time for one displayed minute, t_{measured} seconds (ideal is 60 s)
5. Update ITERATIONS_PER_SECOND using a proportional correction:

$$ITERATIONS_{new} = ITERATIONS_{old} \cdot \left(\frac{t_{measured}}{60} \right)$$

This follows from the idea that the displayed clock speed is inversely proportional to the iteration constant. If one “minute” on our display took longer than 60 seconds, then our second was too long and the iteration constant must be increased (and vice versa).

We repeated this tuning iteratively until the one-minute rollover closely matched 60 seconds. After achieving a good one-minute match, we repeated the same measurement over a longer interval (3 minutes) to reduce human reaction-time error and confirm stability. The final calibrated iteration constant used in this project is:

$$ITERATIONS_PER_SECOND = 2,600,000$$

And how we used this calibrated constant to increment the time correctly is explained in the previous section.

To further characterize our timing function with experimental results we conducted this test to see how our functioned performed within the complete operating conditions:

1. Run clock script
2. Enter manual mode
3. Set time for 10:00 AM
4. As soon as the last 0 is pressed, simultaneously start a timer
5. Stop timer manually as soon as the clock shows: 10:01 AM
6. Record time for 1 minute clock time:

Results:

Manual Clock Characterization	
Trial	Seconds taken
1	59.93
2	59.94
3	59.96
4	60.81
5	59.89

Average	60.106
---------	--------

Figure 5: Manual clock characterization data.

This data gives an expected manual clock minute equal to 60.106 actual seconds.

With this data in mind during the 2-hour clock test, would expect the error to be approximately:

$$\Delta t \cdot 2 \cdot 60 \text{ min} = \text{time dif}$$

$$\Delta t = 60.106 - 60 = \frac{.106s}{min}$$

$$\frac{.106s}{min} \cdot 60 \text{ min} \cdot 2 = 12.72s$$

Theoretical Time Difference over two hours of runtime = 12.72s

Time.Sleep Characterization:

To characterize the Time.Sleep function within the context of the clock program and calculate a theoretical error value, we executed the same procedure as previously described:

1. Run clock script
2. Enter manual mode
3. Set time for 10:00 AM
4. As soon as the last 0 is pressed, simultaneously start a timer
5. Stop timer manually as soon as the clock shows: 10:01 AM
6. Record time for 1 minute clock time:

Results:

Time.Sleep Clock Characterization	
Trial	Seconds taken
1	62.41
2	62.39
3	62.57
4	62.41
5	62.44
Average	62.444

Figure 6: Time.Sleep Characterization data.

The *time.sleep* data gives an expected manual clock minute equal to 62.444 actual seconds.

With this data in mind during the 2-hour clock test, would expect the error to be approximately:

$$\Delta t \cdot 2 \cdot 60 = \text{time diff}$$

$$\Delta t = 62.444s - 60s = \frac{2.444s}{min}$$

$$\frac{2.444s}{min} \cdot 2 \cdot 60 \text{ min} = 293.28s$$

Theoretical Time Difference after 2 hours of runtime = 293.28s

Deliverable 4: Final Timing Function Accuracy (2 Hour Test)

Link To Timelapse: <https://youtu.be/tlYcAgZpxhg?si=BUU38WdTKenSEuUh>

To evaluate the long-term accuracy of our calibrated timing function, we performed a continuous 2-hour test using MANUAL mode.

Test Procedure:

1. Entered MANUAL mode
2. Set the clock to 12:00 PM.
3. At the moment the final minute digit was entered, we started a stopwatch
4. Every 10 minutes of real time, we recorded how many seconds our clock was behind real time.
5. This process was repeated until 120 minutes (2 hours) had elapsed

The recorded data is show in the table below:

Actual Time (minutes)	Clock Delay (seconds behind)
10	3
20	8
30	11
40	15
50	19
60	25
70	31
80	35
90	41
100	47
110	51
120	55

Figure 7: Chart Showing Recorded Data From 2 Hour Test.

So, calculating our timing functions accuracy:

$$\text{Total Drift} = \mathbf{55 \text{ seconds}}$$

$$\text{Drift per minute} = \frac{55 \text{ sec}}{120 \text{ min}} = \mathbf{0.458 \text{ sec/min}}$$

$$\text{Drift per hour} = 0.458 * 60 = \mathbf{27.5 \text{ sec/hour}}$$

$$\text{Percent Error} = \frac{55}{7200} \cdot 100 = \mathbf{0.764\%}$$

Our calibrated timing function has an accuracy of:

$$\mathbf{\pm 0.458 \text{ Seconds/Minute}; \pm 27.5 \text{ Seconds/Hour}; \pm 55 \text{ Seconds per 2-hour period}}$$

The data shows an approximately linear drift over time, indicating consistent systematic timing error rather than random jitter. This suggests that the remaining inaccuracy is primarily due to small calibration error in the `ITERATIONS_PER_SECOND` constant and unavoidable CPU execution overhead variations.

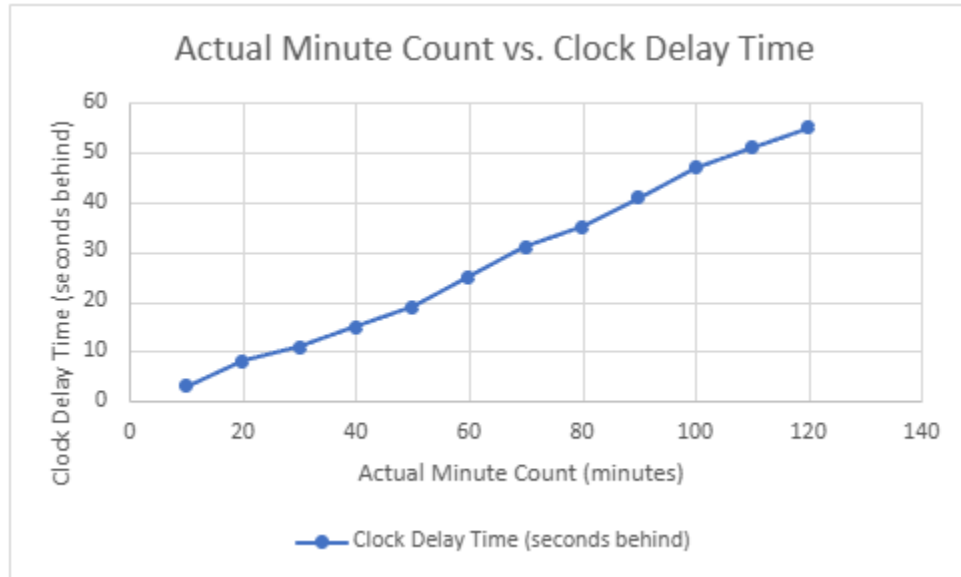


Figure 8: Graph Showing Linear Drift Over Time.

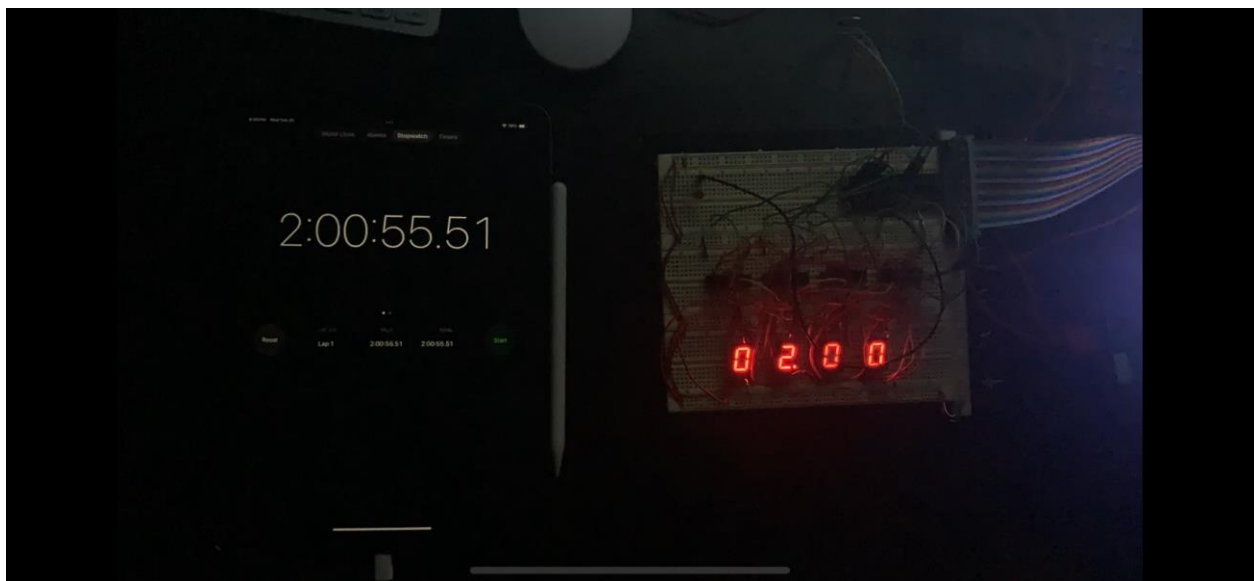


Figure 9: Photo of clock time at end of two-hour test (manual function).

Deliverable 5: Sleep Timing Function Accuracy (2 Hour Test)

Link to timelapse: https://youtu.be/cWtmxaVM_V8?si=PA7Ts-CdjJbIxBL5

To test the accuracy of the time.sleep library over an extended period of time we modified our original manual timing script to increment based on a time.sleep(1) call.

The time.sleep implementation works by pausing program execution for a specified number of seconds before updating the clock variables. In this case, sleep(1) is called to delay the program for approximately one second, and then the manual seconds counter is incremented. When the seconds reach 60, the minutes and hours are updated accordingly.

```
def manual_tick_nonblocking():
    #wait 1 second, then advance manual clock by 1 second
    global manual_s, manual_m, manual_h24, pm

    sleep(1)

    manual_s += 1
    if manual_s >= 60:
        manual_s = 0
        manual_m += 1
        if manual_m >= 60:
            manual_m = 0
            manual_h24 = (manual_h24 + 1) % 24

    # update display once per minute
    h12, pm_flag = convert_24h_to_12h(manual_h24)
    pm = pm_flag
    load_12h_into_memory(h12, manual_m)
```

Figure 10: Time.Sleep timing function.

To conduct the long-term test. We used the same test procedure as for the self-written timing function:

Test Procedure:

1. Entered MANUAL mode
2. Set the clock to 12:00 PM.
3. At the moment the final minute digit was entered, we started a stopwatch
4. Every 10 minutes of real time, we recorded how many seconds our clock was behind real time.
5. This process was repeated until 120 minutes (2 hours) had elapsed

The recorded Data is shown in the table below:

Actual Time (minutes)	Clock Delay (seconds behind)
10	24
20	40
30	64
40	80
50	104
60	120
70	136
80	180
90	234
100	260
110	276
120	295

Figure 11: Chart Showing Recorded Data From 2 Hour Test using Sleep.

So, calculating our timing functions accuracy:

$$\text{Total Drift} = \mathbf{295 \text{ seconds}}$$

$$\text{Drift per minute} = \frac{295 \text{ sec}}{120 \text{ min}} = \mathbf{2.458 \text{ sec/min}}$$

$$\text{Drift per hour} = 2.458 * 60 = \mathbf{147.5 \text{ sec/hour}}$$

$$\text{Percent Error} = \frac{295}{7200} \cdot 100 = \mathbf{4.097\%}$$

The time.sleep() timing function has an accuracy of:

$$\mathbf{\pm 2.458 \text{ Seconds/Minute}; \pm 147.5 \text{ Seconds/Hour}; \pm 295 \text{ Seconds per 2-hour period}}$$

This data also shows relatively linear drift over time, but at a much higher rate than the self-made timing function.

At the end of the test, the Time.Sleep clock is approximately 295 seconds slower than the real clock.

This result matches our expected theoretical drift very well. Whereas our theoretical drift was approximately 293.38 seconds over 2 hours, our actual results drifted by approximately 295 seconds. Although not accurate with reference to a RTC second, this result does speak to the more consistent behavior of the *time.sleep()* function.

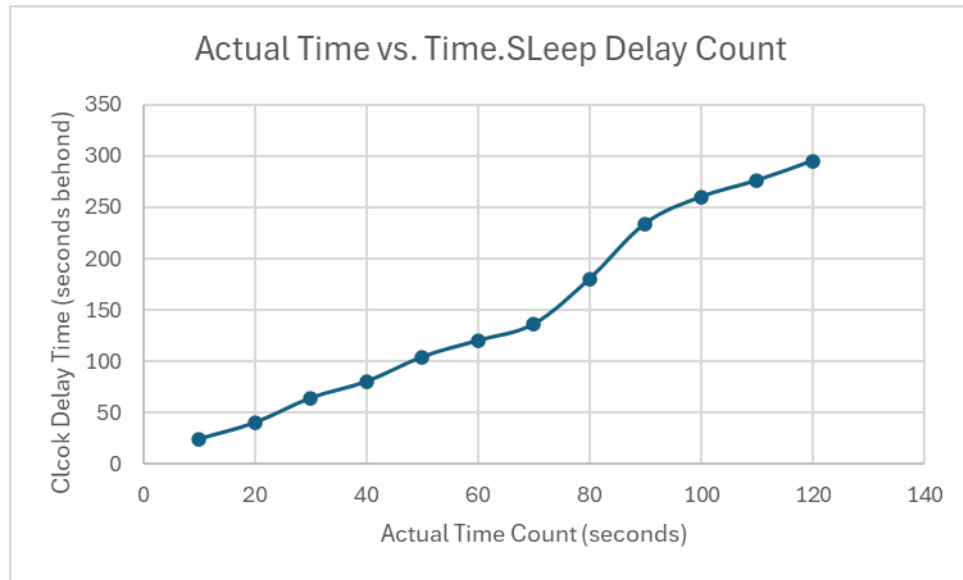


Figure 12: Graph Showing Linear Drift Over Time.

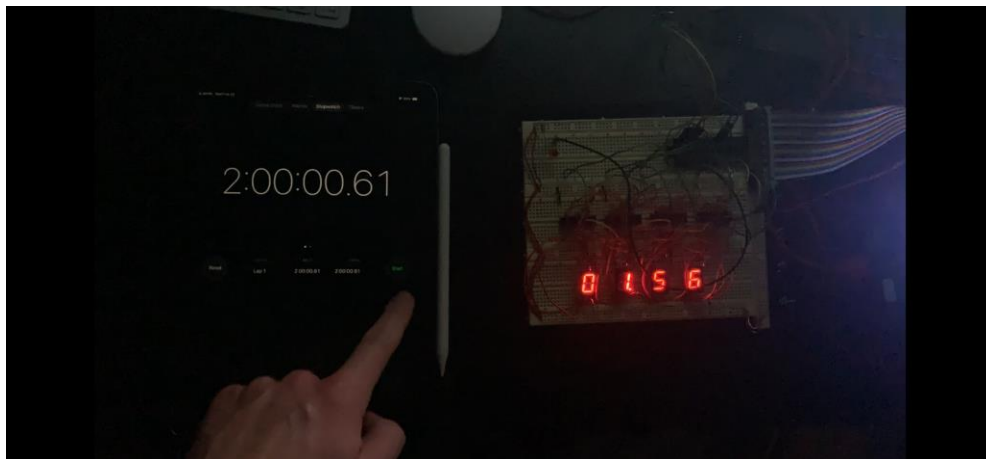


Figure 13: Photo of clock time at end of two-hour test (sleep function).

Deliverable 6: Comparison between Sleep Timing and Our Timing Function

Manual

Our manual timing function uses a software delay loop to approximate one second. It relies on `burn_` iterations and a chunked delay generator to create the timing instead of pausing the program directly. Rather than waiting for a full second all at once, it breaks that second into smaller pieces and checks the keypad during those pieces. That makes it more responsive because it can detect key presses like the BBB reset while the second is still counting. This is an important addition because originally, we were having trouble detecting the BBB sequence while running in manual mode.

In terms of accuracy, the delay is based on calibrated loop counts, so once tuned for a specific processor speed, it can be relatively consistent. However, if the CPU frequency changes or the system is under load, the timing can drift since it depends directly on how fast the processor executes instructions. It also keeps the CPU busy the whole time since it is constantly looping.

Our manually made function results in a more accurate clock time over two hours, however the expected result deviated significantly based on what we calculated theoretically from the timing characterization.

Time.Sleep

The `time.sleep` based version is much simpler. It calls `sleep(1)` and then increments the clock by one second. The operating system handles the timing instead of a manual loop, which is usually more efficient and does not waste CPU cycles. However, sleep is controlled by the OS scheduler, so the actual delay may be slightly longer than one second depending on scheduling and system load. This means it is not perfectly precise and can introduce small timing errors over long periods. The tradeoff is that nothing else runs during the one second sleep, so keypad input is only handled after the pause finishes. Overall, the sleep approach is cleaner and more efficient, while the chunked loop approach gives better responsiveness and more direct control over timing at the cost of complexity and CPU usage.

In contrast to the manually made function, the `time.sleep()` had a greater drift over time compared to the RTC, but this result closely matched what we had expected based on the theoretical calculation from the characterization data.

Direct Comparison Summary:

1) Accuracy Over 2 Hours

Method	Total Drift (2 Hours)	Drift per Hour	Percent Error
Manual timing function	55 seconds	27.5 sec/hour	0.764%
Time.sleep()	295 seconds	147.5 sec/hour	4.097%

Figure 14: Table showing the accuracy over 2-hours for each method.

From the 2-hour test the manual timing function was approximately:

$$\frac{147.5}{27.5} \approx 5.36$$

Times more accurate than the sleep-based implementation.

This shows that, after calibration, the busy-loop timing approach significantly reduced long-term drift compared to *sleep()*.

2) Responsiveness

- Manual timing function: uses chunked iteration blocks and checks keypad input during timing accumulation. This allows detection of sequences like BBB while time is advances.
- Sleep(): Calls sleep(1) and blocks the program for a full second. During this time, keypad input cannot be processed. Input is only handled after the sleep call finishes.

Result: The manual timing function is more responsive and better suited for real-time interaction.

3) Predictability & Control

- Manual timing function: Timing depends on processor execution speed. Once calibrated for a specific CPU, it can be very inconsistent. However, changes in clock frequency or system load affect accuracy.
- Sleep(): Timing depends on OS scheduler and system timing resolution. The delay may be slightly longer than requested due to scheduling latency.

Result: The manual timing function gives direct control over timing behavior, while *sleep()* relies on the operating system's scheduling mechanisms.

4) Final Comparison Statement

Although the *time.sleep()* method is simpler and more CPU-efficient, it exhibited significantly greater long-term drift in our 2-hour accuracy test.

The manually implemented timing function, while more complex and CPU-intensive, achieved:

5.36x better long-term accuracy

And improved system responsiveness by allowing keypad scanning during timing accumulation.

Therefore:

- Manual timing function – More accurate and responsive, higher CPU cost.
- Sleep() function – Simpler and more efficient, but less accurate and less responsive.

Deliverable 7: Causes of Timing Function Errors:

Multiple factors affect the timing results of our manual function. Since the design is based on a calibrated busy loop, the accuracy depends on how long each loop iteration takes on the Raspberry Pi while the full program is running. The processor is also handling GPIO updates, keypad scanning, display refresh, and other background tasks, so the execution time per iteration is not perfectly constant. Any small variation in instruction timing accumulates over long durations. Although we calibrated the iteration constant using one minute and three minute tests, that short characterization window does not fully represent behavior over two hours. From the characterization data, we expected approximately 12.72 seconds of drift over two hours, but the measured result was about 55 seconds off. This indicates that the theoretical iteration constant does not scale linearly over long periods, and that system level effects such as CPU scheduling, and overall program overhead introduce additional error that was not captured during calibration.

Deliverable 8: Main Loop & FSM Description

```
# STATE MACHINE
if state == "IDLE":
    display_memory = [0, 0, 0, 0]
    render_display(pm=False)

    if k == 'A':
        enter_auto()
    elif k == 'B':
        enter_manual()

elif state == "AUTO":
    # When display is off, we still call get_time() to keep globals updated
    if display_on:
        display_real_time()
    else:
        get_time()

elif state == "MANUAL":
    if not manual_done:
        # User is entering time
        manual_entry_step(k)
    if display_on:
        render_display_blink(pm=pm)
    else:
        render_display(pm=False)
```

Figure 15: Code snippet of some of the FSM logic in our main loop.

Our program is structured around a continuously running main loop that implements a finite state machine (FSM). The FSM controls overall clock behavior and determines how keypad input and display updates are handled. The system operates in three state: IDLE, AUTO, and MANUAL.

At each iteration of the main loop, the program first scans for keypad input and checks for global actions (such as display toggling or the BBB reset sequence). After processing any global handlers, the FSM evaluates the current state and executes the corresponding logic.

IDLE State

In the IDLE state, the display is reset to 00:00, which serves as the selection screen. From this state:

- Pressing A transitions the system into AUTO mode.
- Pressing B transitions the system into MANUAL mode.

This state does not advance time; it simply waits for a mode selection.

AUTO State

In Auto mode, the clock displays the real-time system clock retrieved from the operating system. If the display is enabled, *display_real_time()* is called to update the digits. If the display is disabled, *get_time()* is still called to keep internal time variables updated. The AUTO state continuously refreshes the time without requiring additional user interaction.

MANUAL State

The MANUAL state operates in two phases:

1. Time Entry Phase: The user enters a time using the keypad. The currently edited digit blinks to indicate position, and input is validated before being accepted. During this phase, the display is updated using *render_display_blink()* to visually show the blinking digit.
2. Run Phase: Once a valid time is entered, the system transitions to running mode. The manual timing function (*manual_tick_nonblocking*) advances the clock using the calibrated iteration-based delay. The updated time is then rendered to the display each loop cycle.

Overall Loop Behavior

The FSM design ensures that:

- Each state has clearly defined responsibilities
- Transitions occur only when specific keypad inputs are detected
- Timing, display rendering, and keypad scanning are handled within a single continuous loop
- The system remains responsive because no state blocks execution for long periods.

This structured FSM approach keeps the clock logic modular, and easy to extend while maintaining real-time responsiveness.

Deliverable 9: Description of Debouncing Technique

We opted for a simple software fix to debounce the input from our keypad button presses.

```
#debounce: wait for release
while GPIO.input(col) == 1:
    sleep(0.01)
sleep(0.05)
```

Figure 16: Software Debouncing Technique.

This three line statement is located inside the function `get_key_press()` that matches a keypad button press with a unique input character. The 'col' variable represents one of the keypads columns registering high, which is synonymous with a keypad press. While that input is high, the program will sleep which won't allow any other actions to execute/register. It's important that the sleep statement inside the while loop is short so as not to freeze the program long after the button is released. The short sleep statement after the while loop prevents 'bouncing inputs' caused by the mechanics of the switch from registering as additional inputs.

We opted for a software debounce technique due to its simplicity. We considered the hardware technique of adding capacitors to each keypad column input, but since the hardware complexity for this project was already extensive, with a large number of components and connections, we decided that it would be preferable to shift some of this complexity to the software side or the project.

To determine how to implement the debouncing technique we thought about what behavior we wanted during a keypad press:

- Only register a single press in a short time period
- Eliminate 'bouncing inputs'
- Be responsive enough on the release to detect next press

Deliverable 10: Most Challenging Part of The Clock Project

The most challenging part of developing the fully functional seven segment display clock was the software integration process following checkpoint 4. Checkpoint 4 required restructuring the code to support multiple operating states while maintaining reliable keypad input, display rendering, and timekeeping. The software integration requires the creation of new functions and modifications of pre-existing functions. This allowed for correct operation within the updated main loop and state machine. A significant challenge was ensuring that the newly implemented states and features do not interfere with one another. This is because the code relies heavily on shared global variables resulting in small changes in one function often affecting other portions of the program. The solution to this required debugging, restructuring of logic flow, and repeated testing to maintain consistent predictable behavior across system operation.

As a team, we overcame these challenges through consistent communications, shared debugging sessions, and technical perseverance. Rather than splitting development tasks, we collaboratively reviewed errors and walked through the code execution step by step to identify errors in logic flow. Each member contributed to function development and modifications, error handling, and software integrated testing to verify the state machine logic. This approach improved the workflow throughout the deliverable and allowed for successful integration that met all necessary features.

References

1. Dr. Herring GitHub instructions: [Python/Seven Segment Display Clock & Timer at main · Herring-UGAECSE-4230-S26/Python](#)
2. PI GPIO Documentation: [pigpio library](#)
3. Rpi GPIO Documentation: [RPi.GPIO · PyPI](#)
4. Time Library Documentation: [time — Time access and conversions — Python 3.14.3 documentation](#)
5. Raspberry PI Documentation: [Raspberry Pi 4 Model B specifications – Raspberry Pi](#)
6. SSD Pinout: [How to Interface a Seven-Segment Display with an Arduino](#)
7. D-Type Flip-Flop datasheet: <https://www.ti.com/lit/gpn/CD74HCT574>

AI References (Model, Prompt)

1. ChatGPT, “organize this code and remove unused portions”
2. ChatGPT, “If we are making a manual timing library without time.sleep is it better to time it to count one second or one minute”
3. Google Gemini, “can you read this code and help us understand why the BB function is not working when the manual time mode is initiated”
4. Google Gemini, “What would be the next best step for our code to be able to enter the time manually or four seven segment displays”
5. Google Gemini, “For what reasons might a 1 second manual timing function not be accurate over two hours, while running on a raspberry pi”
6. ChatGPT, “Does RP4 have enough output current to support a seven-segment display”
7. Google Gemini, “Read this code and provide a suggestion as to how we can toggle the display using the pound key, but we want to ensure that the display is restored to the previous value”
8. Google Gemini, “Can GPIO 1 on the raspberry pi 4 b turn power and LED as a GPIO output”
9. ChatGPT, “Fix the indentation error in this code”